AMBA family & AHB architecture

01

Where AHB sits in an SoC, what blocks make up an AHB system, and why its design choices (pipelining, single clock edge, mux-based bus) exist.

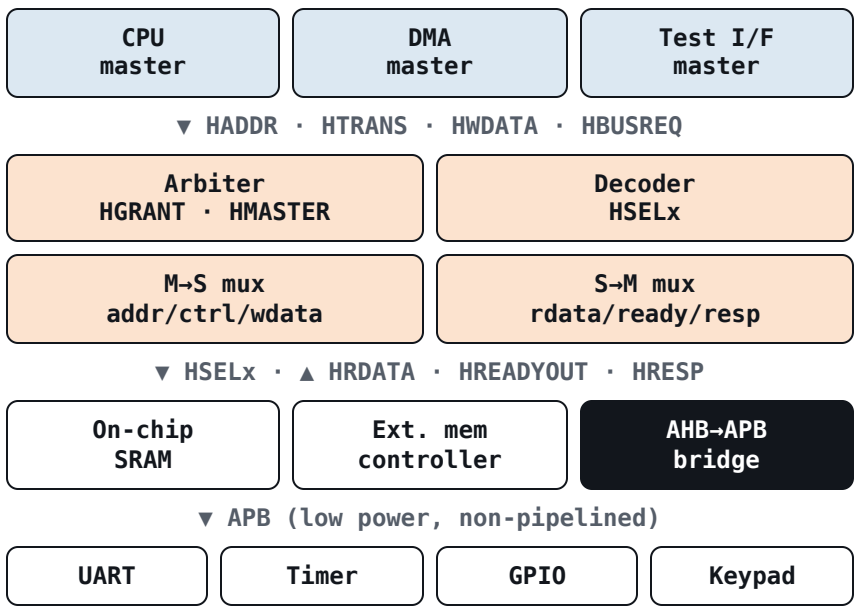
CORE IDEA

AHB = Advanced High-performance Bus

- **Synchronous, pipelined, on-chip bus** for high-bandwidth blocks: CPU, DMA, on-chip SRAM, external memory controller.
- Introduced in **AMBA 2 (1999)** to replace ASB, whose bidirectional (tristate) bus and two-edge timing did not scale with frequency.
- Every transfer = **address phase** (1 cycle) + **data phase** (1+ cycles). Address of transfer N+1 overlaps data of transfer N → up to **one transfer per clock**.
- **Single clock edge**: everything sampled on rising `HCLK` → easy STA, synthesis and scan.
- **Non-tristate**: separate `HWDATA` and `HRDATA` buses steered by multiplexers, not shared wires.
- Supports **bursts** (4/8/16-beat incrementing & wrapping, plus undefined INCR), **wait states**, **error responses**, and in AHB2 **multi-master arbitration** with **SPLIT/RETRY**.
- Data bus width: 8 to 1024 bits (32 bit is most common).

SYSTEM VIEW

Typical AMBA 2 SoC



TIMELINE — KNOW THE GENERATION OF EACH SIGNAL

Generation	Year	What it introduced	Interview hook
AMBA 1	1996	ASB (system bus), APB (peripheral bus)	ASB used tristate bidirectional data — timing pain
AMBA 2	1999	AHB: pipelining, bursts, multi-master arbiter, SPLIT/RETRY, 2-bit HRESP	This playlist's core — "full" AHB
AMBA 3	2003	AXI3, AHB-Lite (single master), APB3 (PREADY, PSLVERR)	AHB-Lite drops arbiter, SPLIT, RETRY; HRESP becomes 1 bit
AMBA 4	2010	AXI4, AXI4-Lite, AXI4-Stream, ACE, APB4 (PPROT, PSTRB)	Coherency arrives (ACE)
AMBA 5	2013+	CHI, AHB5 (TrustZone HNONSEC, exclusive access, user signals), AXI5	AHB5 targets Cortex-M23/M33 secure systems

COMPARE

APB vs AHB vs AXI

Feature	APB	AHB	AXI
Pipelined	No	Yes (A/D)	Yes (channels)
Min cycles/xfer	2	1 (overlapped)	1
Bursts	No	Yes	Yes (1–256)
Outstanding	No	1 in data phase	Many (IDs)
Out-of-order	No	No	Yes
Read+write in parallel	No	No	Yes
Multi-master	No	Arbiter / multi-layer	Interconnect
Used for	Registers	MCU system bus	High-end SoC

DEEP DIVE

Why each design choice

- **Pipelining** hides address decode latency: the slave decodes the next address while the current data moves.
- **Mux, not tristate**: no bus contention, no turnaround cycles, works in FPGA and standard-cell flows, testable by scan.
- **Split read/write data buses**: write→read back-to-back needs no dead cycle.
- **HREADY wait states** let slow slaves stretch only the data phase, without a separate handshake.
- **Bursts** give the slave address predictability (SEQ = previous + size), so memories can pre-fetch.
- **1 KB burst boundary** guarantees a burst stays inside one slave (smallest decode region is 1 KB).

TRAPS INTERVIEWERS SET

- "AHB is full duplex" — **no**. Two data buses exist, but only one transfer is in its data phase at a time.
- "AHB supports outstanding transactions" — only **one** address phase overlaps **one** data phase; no IDs, no reordering.
- "AHB-Lite = AHB" — Lite is **single-master**; multi-master needs a multi-layer interconnect.
- `HRESETn` is the **only active-low** AHB signal; reset is asynchronous assert, synchronous deassert in practice.
- Slaves never drive the bus directly: every slave output goes through the **S→M mux**.

INTERVIEW RAPID-FIRE

Why would you choose AHB over AXI?
Lower area and power, simpler verification, enough bandwidth for MCU-class systems (Cortex-M).

What limits AHB throughput?
Single in-flight data phase: a slow slave stalls the whole bus (head-of-line blocking).

What does "single clock edge" buy you?
Half-cycle paths vanish; timing closure and DFT are straightforward.

Where does APB fit?
Behind an AHB-to-APB bridge (an AHB slave) for low-bandwidth register access.

Minimum blocks in an AHB system?
Master, slave(s), decoder, S→M mux; plus arbiter and M→S mux when multi-master.

Max transfer size?
Equal to data bus width: `HSIZE` must never exceed it.

Remember: address now, data next cycle — one edge, muxes not wires, many slaves.

AHB signal map

Every signal, who drives it, which phase it belongs to, and the encodings (HSIZE, HPROT, byte lanes) that interviewers ask you to recite.

02

MASTER TABLE — ORANGE = AHB2 MULTI-MASTER ONLY (GONE IN AHB-LITE)

Signal	Width	Driver	Phase	Meaning	Interview note
HCLK	1	Clock src	—	Bus clock	All timing on rising edge
HRESETn	1	Reset ctrl	—	Active-low reset	Only active-low signal; masters drive IDLE
HADDR	32	Master	Addr	Byte address	Must be aligned to HSIZE
HTRANS	2	Master	Addr	IDLE / BUSY / NONSEQ / SEQ	Only NONSEQ/SEQ are real transfers
HWRITE	1	Master	Addr	1 = write, 0 = read	Constant across a burst
HSIZE	3	Master	Addr	Bytes per beat = 2^HSIZE	≤ data bus width, constant in burst
HBURST	3	Master	Addr	Burst type and length	SINGLE, INCR, WRAP4/8/16, INCR4/8/16
HPROT	4	Master	Addr	Protection / cache attributes	Unknown → drive 4'b0011
HWDATA	8–1024	Master	Data	Write data	One cycle after HADDR; held while HREADY low
HSELx	1	Decoder	Addr	Slave select	Combinational from HADDR
HRDATA	8–1024	Slave→mux	Data	Read data	Valid only when HREADY high
HREADYOUT	1	Slave	Data	This slave's ready	Goes into the S→M mux
HREADY	1	S→M mux	Data	Bus-wide "transfer done"	Input to every master AND slave
HRESP	2 (AHB2)	Slave	Data	OKAY/ERROR/RETRY/SPLIT	1 bit in AHB-Lite/AHB5
HBUSREQx	1	Master	—	Request the bus	One per master
HLOCKx	1	Master	Addr	Request a locked sequence	Asserted with HBUSREQ, before address
HGRANTx	1	Arbiter	—	Master owns next addr phase	Ownership = HGRANT & HREADY high
HMASTER	4	Arbiter	Addr	ID of owning master	Mux select; slaves record it for SPLIT
HMASTLOCK	1	Arbiter	Addr	Current transfer is locked	Kept in AHB-Lite (driven by master)
HSPLITx	16	Slave	—	Which split masters may retry	One bit per master, to arbiter

HSIZE ENCODING

HSIZE	Bytes	Bits	Name
000	1	8	Byte
001	2	16	Halfword
010	4	32	Word
011	8	64	Doubleword
100	16	128	4-word line
101	32	256	8-word line
110	64	512	—
111	128	1024	—

Alignment rule: `HADDR % 2^HSIZE == 0`.

HPROT[3:0] BITS

Bit	0 means	1 means
[0]	Opcode fetch	Data access
[1]	User access	Privileged access
[2]	Non-bufferable	Bufferable
[3]	Non-cacheable	Cacheable

Masters with no protection info drive `4'b0011` (data, privileged, non-bufferable, non-cacheable). Most slaves ignore HPROT; MPUs and memory controllers use it. AHB5 widens it to 7 bits.

BYTE LANES, 32-BIT LITTLE-ENDIAN

Access	HADDR[1:0]	Active lanes
Byte	00 / 01 / 10 / 11	[7:0] / [15:8] / [23:16] / [31:24]
Halfword	00 / 10	[15:0] / [31:16]
Word	00	[31:0]

GROUP BY PHASE

What is sampled when

ADDRESS PHASE
HADDR · HTRANS
HWRITE · HSIZE
HBURST · HPROT
HSELx · HMASTER
HMASTLOCK

DATA PHASE
HWDATA (master)
HRDATA (slave)
HREADY / HREADYOUT
HRESP

A slave samples address/control on the rising edge where `HSELx & HREADY & HTRANS[1]` are all high. Data-phase signals belong to the **previous** address.

TRAPS

- **HREADY ≠ HREADYOUT.** HREADYOUT is a slave's output; HREADY is the muxed bus value every slave must watch.
- HSELx alone is not enough — a slave must qualify with **HREADY**, or it may latch an address while another slave's transfer is still waiting.
- Write data lags address by one cycle: `HWDATA` at cycle N+1 belongs to `HADDR` at cycle N.
- Narrow transfers: master drives only active lanes meaningfully; slave returns valid data only on active lanes.
- HMASTER changes with the address phase; the HWDATA mux needs a **registered** copy of it.

INTERVIEW RAPID-FIRE

Why is HREADY an input to slaves?
So a selected slave knows when the previous transfer (maybe to another slave) finished and its own address phase is real.

Can a 32-bit bus do a 64-bit HSIZE transfer?
No. HSIZE must be ≤ bus width; a 64-bit value needs two word transfers or a burst.

Which signals must stay constant through a burst?
HWRITE, HSIZE, HBURST, HPROT. Only HADDR (and HTRANS) advance.

What's HPROT used for in practice?
MPU checks (privileged vs user), cache/buffer policy at memory controllers, trace.

Remember: H-signals with "x" are per-slave or per-master; HREADY is the bus heartbeat.

Transfers, pipelining & HTRANS

03

How a read and a write move through address and data phases, how they overlap, and the four transfer types that tell a slave whether a cycle is real.

TIMING 1 — THREE SINGLE TRANSFERS, ZERO WAIT STATES (WRITE A, READ B, WRITE C)

Signal	T1	T2	T3	T4	T5
HTRANS	NONSEQ	NONSEQ	NONSEQ	IDLE	IDLE
HADDR	A	B	C	–	–
HWRITE	1	0	1	–	–
HWDATA	–	D(A)	–	D(C)	–
HRDATA	–	–	D(B)	–	–
HREADY	1	1	1	1	1
Bus doing	A addr	A data · B addr	B data · C addr	C data	idle

Values show what is on the bus during each cycle; they are sampled on the rising edge that ends the cycle. Blue = address phase, orange = data phase. Three transfers complete in four cycles — the pipeline gives ~1 transfer/clock.

HTRANS[1:0]

Code	Type	Meaning	Slave must
00	IDLE	No transfer wanted	Ignore; OKAY, zero wait
01	BUSY	Mid-burst pause; address/control already shows next beat	Ignore; OKAY, zero wait
10	NONSEQ	Single transfer or first beat of a burst; address unrelated to previous	Perform transfer
11	SEQ	Next beat; addr = prev + 2^HSIZE (or wrapped), control identical	Perform transfer

`HTRANS[1]` = "this is a real transfer" — slaves often decode just that bit.

LEGAL SEQUENCES

- **IDLE → NONSEQ** starts any transfer.
- **NONSEQ → SEQ / BUSY** only if HBURST ≠ SINGLE.
- **BUSY → SEQ / BUSY** inside a burst.
- **BUSY → IDLE / NONSEQ** only to end an undefined-length INCR burst.
- **Illegal:** IDLE → SEQ, IDLE → BUSY, SINGLE NONSEQ → SEQ, fixed burst ending on BUSY.
- During a wait state, a master may change **IDLE → NONSEQ**; after that the transfer is held until HREADY goes high.

TIMING 2 — INCR4 WORD WRITE BURST WITH A BUSY BEAT

Signal	T1	T2	T3	T4	T5	T6
HTRANS	NONSEQ	SEQ	BUSY	SEQ	SEQ	IDLE
HADDR	0x20	0x24	0x28	0x28	0x2C	–
HBURST	INCR4	INCR4	INCR4	INCR4	INCR4	–
HWDATA	–	D(0x20)	D(0x24)	ignored	D(0x28)	D(0x2C)
HREADY	1	1	1	1	1	1

During BUSY the master already shows the **next** beat's address (0x28). The slave ignores the BUSY cycle and its data phase, then takes the SEQ at 0x28 in T4.

READ VS WRITE MECHANICS

- **Write:** master drives HWDATA in the data phase and **holds it** until HREADY is sampled high.
- **Read:** slave drives HRDATA; master captures it on the edge where HREADY is high. Minimum read latency = 2 cycles (address + data).
- **Write → read back-to-back:** no turnaround cycle, because HWDATA and HRDATA are separate buses.
- While HREADY is low, the next transfer's address and control are **frozen** (address phase is extended).
- After reset, masters drive **IDLE** until they need the bus.

TRAPS

- "BUSY is a wait state from the master" — only **inside bursts**; it is not a general stall.
- Data shown during IDLE/BUSY cycles is **don't-care** — scoreboards must not compare it.
- Pairing bug: a monitor that samples HWDATA in the same cycle as HADDR is off by one beat.
- HTRANS sampled when HREADY is low is not yet "accepted" — only the edge with HREADY high counts.
- Slave responding with wait states to IDLE/BUSY is a **protocol violation**.

SVA YOU SHOULD BE ABLE TO WRITE

```
// address/control held while the bus is waited
property p_ctrl_stable;
  @(posedge HCLK) disable iff (!HRESETn)
  (HTRANS[1] && !HREADY) |=>
    $stable({HADDR, HWRITE, HSIZE, HBURST, HTRANS});
endproperty

// no SEQ/BUSY right after an accepted IDLE
property p_no_seq_after_idle;
  @(posedge HCLK) disable iff (!HRESETn)
  (HREADY && HTRANS == 2'b00) |=>
    !(HTRANS inside {2'b01, 2'b11});
endproperty

// write data stable while waited (wr_dphase = registered write)
property p_wdata_stable;
  @(posedge HCLK) disable iff (!HRESETn)
  (wr_dphase && !HREADY) |=> $stable(HWDATA);
endproperty
```

INTERVIEW RAPID-FIRE

Why does BUSY exist?
A master (e.g. DMA) can't supply the next beat yet but wants to keep the burst — the slave can still prefetch.

How many transfers can be in flight?
Two: one in address phase, one in data phase.

Can the address phase take more than one cycle?
Yes — it is stretched whenever the previous data phase is waited.

What does a slave do with IDLE while selected?
Nothing; zero-wait OKAY.

Remember: HTRANS[1] = real. NONSEQ opens, SEQ continues, BUSY pauses, IDLE rests.

Bursts & address calculation

04

HBURST types, incrementing vs wrapping address math, the 1 KB boundary rule and early termination — the most-asked whiteboard topic in AHB rounds.

HBURST[2:0]

Code	Type	Beats
000	SINGLE	1
001	INCR	Undefined (≥1)
010	WRAP4	4
011	INCR4	4
100	WRAP8	8
101	INCR8	8
110	WRAP16	16
111	INCR16	16

Pattern: odd codes (except 001) are INCRx, even codes ≥ 010 are WRAPx. Beats count **transfers**, not bytes.

ADDRESS MATH

Two formulas to memorise

```
size  = 2^HSIZE                      // bytes per beat
INCR  : next = addr + size

B     = beats * size                 // wrap boundary (bytes)
WRAP  : next = (addr & ~(B-1)) | ((addr + size) & (B-1))
```

- Every beat's address is **aligned to HSIZE** — bursts included.
- Wrapping bursts wrap at a boundary of **beats × size**: WRAP4 word → 16 B, WRAP8 word → 32 B, WRAP16 word → 64 B.
- A wrapping burst never leaves its aligned block, so it never crosses 1 KB.
- HWRITE, HSIZE, HBURST, HPROT are constant across all beats.

WORKED EXAMPLES — ADDRESS OF EVERY BEAT

Burst · size · start	B1	B2	B3	B4	B5	B6	B7	B8
INCR4 · word · 0x38	0x38	0x3C	0x40	0x44	–	–	–	–
WRAP4 · word · 0x38 (B=16)	0x38	0x3C	0x30	0x34	–	–	–	–
WRAP8 · word · 0x34 (B=32)	0x34	0x38	0x3C	0x20	0x24	0x28	0x2C	0x30
WRAP8 · half · 0x34 (B=16)	0x34	0x36	0x38	0x3A	0x3C	0x3E	0x30	0x32
INCR8 · byte · 0x3FE	0x3FE	0x3FF	**0x400**	**…**	**ILLEGAL**	**crosses**	**1 KB**	**boundary**
INCR4 · word · 0x3F8	0x3F8	0x3FC	**0x400**	**0x404**	**ILLEGAL — split into two bursts**			

Orange = the wrap point. WRAP16 byte from 0x07 runs 0x07…0x0F then 0x00…0x06 (B = 16).

THE 1 KB RULE

- Incrementing bursts (INCR, INCR4/8/16) must **not cross a 1 KB address boundary**.
- Reason: 1 KB is the minimum address space a decoder allocates to a slave, so a burst that stays inside 1 KB always targets one slave — the decoder never switches HSEL mid-burst.
- Check: `(start & ~0x3FF) == (last & ~0x3FF)`.
- A master with a long INCR that reaches 0x…400 must end the burst and restart with a new **NONSEQ**.

EARLY TERMINATION

- **AHB2:** arbiter may take the grant away mid-burst (INCR especially). The master re-arbitrates and resumes the rest as **NONSEQ + INCR** (or singles).
- **ERROR response:** master may cancel remaining beats (drive IDLE) or continue.
- **Slaves must cope**: seeing NONSEQ or IDLE where a SEQ was expected is legal — never assume the burst completes.
- **INCR undefined length** can end at any beat; a 1-beat INCR is legal.
- AHB-Lite has no arbiter, so fixed bursts end early only after ERROR.

CONSTRAINED-RANDOM BURST ITEM

```
rand bit [31:0] haddr;  rand bit [2:0] hsize, hburst;
rand int unsigned beats;

constraint c_size  { (8 << hsize) <= DATA_W; }
constraint c_align { haddr % (1 << hsize) == 0; }
constraint c_beats {
  hburst inside {WRAP4, INCR4}   -> beats == 4;
  hburst inside {WRAP8, INCR8}   -> beats == 8;
  hburst inside {WRAP16, INCR16} -> beats == 16;
  hburst == SINGLE               -> beats == 1;
  hburst == INCR                 -> beats inside {[1:32]};
}
constraint c_1kb {   // incrementing bursts stay in one 1 KB page
  hburst inside {INCR, INCR4, INCR8, INCR16} ->
    (haddr[9:0] + beats * (1 << hsize)) <= 1024;
}
```

TRAPS

- Wrap boundary depends on **HSIZE** — WRAP4 byte wraps at 4 B, not 16 B.
- "WRAP bursts must start at the boundary" — **no**, any aligned address; INCRx starting at an unaligned-to-B address is fine too.
- BUSY beats don't count toward the burst length.
- Scoreboards that predict 4 beats for INCR4 must still handle early termination after ERROR / grant loss.
- Address for SEQ during BUSY is the **next** beat's address, unchanged until SEQ.

INTERVIEW RAPID-FIRE

Why do wrapping bursts exist?

Cache line fills with critical-word-first: fetch the missed word first, then wrap to fill the rest of the line.

What is the largest single burst on a 32-bit AHB?

INCR16 word = 64 bytes (an INCR of undefined length can be longer, inside 1 KB).

Can a slave tell a burst's length up front?

For fixed bursts yes (HBURST); for INCR no — it only sees SEQ beats until something else appears.

Next address of WRAP8 halfword after 0x3E?

B = 16 → 0x30.

Remember: wrap boundary = beats × size; INCR never crosses 1 KB.

Wait states & HREADY

05

How a slow slave stretches the data phase, why every slave watches the muxed HREADY, and what freezes on the bus while it is low.

TIMING 1 — WRITE A WITH 2 WAIT STATES, THEN READ B

Signal	T1	T2	T3	T4	T5
HTRANS	NONSEQ	NONSEQ	NONSEQ	NONSEQ	IDLE
HADDR	A	B	B (held)	B (held)	–
HWRITE	1	0	0	0	–
HWDATA	–	D(A)	D(A) held	D(A) held	–
HREADY	1	**0**	**0**	1	1
HRDATA	–	–	–	–	D(B)
Event	A addr	wait 1	wait 2	A done · B accepted	B done

A's data phase lasts 3 cycles (1 + 2 waits). B's address phase is stretched from T2 to T4 — the pipeline stalls as a whole.

TIMING 2 — WHY HREADY IS AN INPUT TO SLAVES (A → SLAVE S1 WAITS, B → SLAVE S2)

Signal	T1	T2	T3	T4
HADDR	A (S1)	B (S2)	B (S2)	C
HSEL_S1 / HSEL_S2	1 / 0	0 / 1	0 / 1	0 / 0
HREADYOUT_S1	1	**0**	1	1
HREADYOUT_S2	1	1	1	1
S→M mux selects	prev	S1	S1	S2
HREADY (bus)	1	**0**	1	1
S2 action	–	**must NOT sample B**	samples B	B data phase

S2 is selected in T2, but S1 is still busy. If S2 looked only at its own HSEL it would start B a cycle early. Rule: **sample address when HSELx & HREADY & HTRANS[1]**.

RULES

- A slave inserts waits by driving **HREADYOUT low** during its data phase; the transfer completes on the edge where HREADY is high.
- While HREADY is low, the master **holds** HWDATA and the pending address/control; HRDATA is valid only in the final cycle.
- Spec **recommends ≤ 16 wait states** per transfer so one slave can't hog the bus; longer latency → SPLIT/RETRY (AHB2).
- Slave must answer IDLE/BUSY with **zero-wait OKAY** and keep HREADYOUT high when not in a data phase (and out of reset).
- A master **cannot abort** a waited transfer; only an ERROR/RETRY/SPLIT response ends it differently.
- Unmapped addresses go to a **default slave** so HREADY never hangs.

HOW HREADY IS BUILT

- Decoder selects a slave in the **address phase**; that select is **registered** (when HREADY high) to drive the S→M mux in the **data phase**.
- `HREADY = HREADYOUT[sel_q]`, also feeding HRDATA and HRESP from the same slave.
- When no slave is in a data phase (after IDLE), the mux selects a source that returns HREADY = 1.
- Each slave sees **both** its own HREADYOUT and the global HREADY: one says "I'm done", the other says "the bus moved".

HADDR → decoder → **HSEL flop (en = HREADY)** → **S→M mux → HREADY**

TRAPS

- HREADY low does **not** mean "the slave isn't selected" — it means "previous transfer not finished".
- Read data sampled during a wait cycle is garbage — monitors must sample only when HREADY = 1.
- A slave that holds HREADYOUT low while unselected will hang the bus after the next transfer to it.
- Combinational HREADYOUT → HREADY → master → HTRANS loops are a classic timing/STA issue in interconnects.
- Waits on a BUSY beat are illegal (BUSY = zero-wait OKAY).

CHECKS & COVERAGE

```
// bus must not stay waited forever (bound per your slaves)
a_no_hang: assert property (@(posedge HCLK) disable iff (!HRESETn)
  not (!HREADY [*17]));

// IDLE/BUSY in data phase get zero-wait OKAY
a_idle_okay: assert property (@(posedge HCLK) disable iff (!HRESETn)
  (HREADY && HSEL && !HTRANS[1]) |=> (HREADYOUT && HRESP == OKAY));

covergroup cg_wait @(posedge HCLK iff xfer_done);
  cp_waits : coverpoint wait_cnt { bins zero = {0}; bins one = {1};
             bins few = {[2:4]}; bins many = {[5:16]}; }
  cp_beat  : coverpoint beat_pos;   // enum {FIRST, MID, LAST}
  x_rw     : cross cp_waits, hwrite, cp_beat;
endgroup
```

INTERVIEW RAPID-FIRE

Who can insert wait states — master or slave?

Only the slave (via HREADYOUT). A master's only "delay" tool is BUSY inside a burst or IDLE between transfers.

Effect of one wait state on throughput?

The whole pipeline stalls one cycle — address phase of the next transfer is extended too.

What does the default slave return?

Zero-wait OKAY for IDLE/BUSY; two-cycle ERROR for NONSEQ/SEQ.

How do you test a slave's wait-state handling?

Randomise back-to-back transfers across two slaves with random waits; check address isn't taken while HREADY = 0.

Remember: HREADYOUT = "I'm done"; HREADY = "the bus moved". Sample only on HREADY = 1.

Slave responses: HRESP

06

OKAY, ERROR, RETRY, SPLIT — what each means, why non-OKAY responses take two cycles, and what the master must do next.

HRESP[1:0] (AHB2)

Code	Response	Cycles	Master must
00	OKAY	1	Proceed (with HREADY low = wait)
01	**ERROR**	**2**	**May cancel or continue the burst; report fault**
10	RETRY	2	Re-issue the same transfer; keeps arbitrating
11	SPLIT	2	Re-issue when re-granted; arbiter masks it meanwhile

AHB-Lite / AHB5: `HRESP` is **1 bit** (0 OKAY, 1 ERROR). RETRY and SPLIT are removed; AHB5 adds `HEXOKAY` for exclusives.

DECODE HREADY + HRESP TOGETHER

HREADY	HRESP	Meaning
0	OKAY	Wait state
1	OKAY	Done, success
0	ERR/RTY/SPL	1st response cycle
1	ERR/RTY/SPL	2nd cycle — complete

A slave may insert waits (OKAY + HREADY low) **before** starting the two-cycle response.

TIMING — ERROR ON READ A; MASTER CANCELS PENDING B

Signal	T1	T2	T3	T4	T5
HTRANS	NONSEQ	NONSEQ	NONSEQ	IDLE (B cancelled)	IDLE
HADDR	A	B	B	–	–
HREADY	1	0	**0**	**1**	1
HRESP	OKAY	OKAY (wait)	**ERROR**	**ERROR**	OKAY
Meaning	A addr	A waited	error cycle 1	error cycle 2 · A ends	bus idle

The master sees ERROR at the end of T3 while B is still un-accepted (HREADY low). In T4 it may replace B with IDLE — that is the whole reason for the two-cycle response.

DEEP DIVE: WHY TWO CYCLES?

- AHB is pipelined: when a slave decides "error", the **next address is already on the bus**.
- **Cycle 1** (HREADY low): announces the response while the next address phase is still frozen.
- **Cycle 2** (HREADY high): completes the transfer. The master has had one cycle to change HTRANS to IDLE or re-drive the retried address.
- Without it, the next transfer would be accepted before the master could react — breaking RETRY/SPLIT, which must re-issue the **same** transfer first.
- HRESP must stay the same in both cycles.

RETRY VS SPLIT

	RETRY	SPLIT
Arbiter involved?	No	Yes (masks master)
Master priority	Unchanged	Removed until HSPLIT
Bus while waiting	Master keeps retrying	Free for others
Slave tracking	None	Records HMASTER
Starvation risk	Lower-priority masters	Low
Typical use	Short, unknown delay	Long latency (ext. memory)

TRAPS

- "ERROR terminates the burst" — **not automatically**; the master decides. The slave must handle either.
- A single-cycle ERROR (HREADY high on first cycle) is a **protocol violation**.
- Write data of an errored write is not committed — scoreboards must **drop** it, not compare.
- OKAY is the only valid response to IDLE and BUSY.
- RETRY with a fixed-priority arbiter can starve lower-priority masters forever.

ASSERTIONS

```
// non-OKAY response is always two cycles, same value
a_two_cycle: assert property (@(posedge HCLK) disable iff (!HRESETn)
  (HRESP != OKAY && !HREADY) |=>
    (HREADY && HRESP == $past(HRESP)));

// a completing non-OKAY must have had its first cycle
a_first_cycle: assert property (@(posedge HCLK) disable iff (!HRESETn)
  (HREADY && HRESP != OKAY) |->
    ($past(!HREADY) && $past(HRESP) == HRESP));

// after RETRY, master re-issues the same address next
a_retry_reissue: assert property (@(posedge HCLK) disable iff (!HRESETn)
  (HREADY && HRESP == RETRY) |=>
    (HTRANS == NONSEQ && HADDR == retry_addr_q));
```

INTERVIEW RAPID-FIRE

When does a slave return ERROR?

Unmapped address (default slave), write to read-only region, privilege violation via HPROT, unsupported size.

Can an ERROR come after wait states?

Yes: any number of OKAY waits, then the two-cycle ERROR.

How does the CPU learn about an ERROR?

The master turns it into a bus fault / data abort exception.

Why were RETRY and SPLIT dropped in AHB-Lite?

Single master: no one else can use the bus, so freeing it gains nothing; waits are enough.

Remember: OKAY in one, everything else in two — first cycle warns, second completes.

Arbitration & bus handover

07

Multi-master AHB2: request, grant, ownership, HMASTER, default master and locked transfers — and why the data bus changes owner one cycle after the address bus.

FLOW

Request → grant → own

1. Master asserts `HBUSREQx` (and `HLOCKx` for an atomic sequence).
2. Arbiter samples requests every rising edge and asserts one `HGRANTx` using its scheme.
3. Master **owns the address bus** on the edge where `HGRANTx = 1 & HREADY = 1`, and drives NONSEQ next cycle.
4. Arbiter drives `HMASTER` with the owner's ID (M→S mux select; slaves record it for SPLIT) and `HMASTLOCK` if locked.
5. Data bus (HWDATA) follows one cycle later using a **registered HMASTER**.

KEY FACTS

- Up to **16 masters** (HMASTER[3:0]).
- Arbitration **algorithm is not defined** by the spec: fixed priority, round-robin, weighted, TDMA are all legal.
- **Default master**: granted when nobody requests; must drive IDLE (keeps the bus defined).
- **Dummy master**: granted when every requester is SPLIT; only drives IDLE.
- Arbiter may change grant during an **INCR** burst anytime; for fixed-length bursts it usually waits, but the spec allows early termination — the master re-requests and finishes with NONSEQ + INCR.
- A master granted but with nothing to do must drive **IDLE**.

TIMING — HANDOVER FROM M1 (ENDING A BURST) TO M2

Signal	T1	T2	T3	T4
HBUSREQ_M2	1	1	1	1
HGRANT_M1 / M2	1 / 0	0 / 1	0 / 1	0 / 1
HREADY	1	1	1	1
HMASTER	M1	M1	M2	M2
HTRANS · HADDR	SEQ · A+8 (M1)	SEQ · A+C (M1, last)	NONSEQ · B (M2)	SEQ · B+4 (M2)
HWDATA from	M1 · D(A+4)	M1 · D(A+8)	M1 · D(A+C)	M2 · D(B)

Grant moves to M2 in T2; M2 samples `HGRANT & HREADY` high at the end of T2 and takes the **address** bus in T3. M1 still owns the **data** bus in T3 to finish its last beat — ownership of data lags address by exactly one transfer.

LOCKED TRANSFERS

- Master asserts `HLOCKx` together with HBUSREQx, at least one cycle before the first locked address.
- Arbiter drives `HMASTLOCK` during the locked address phases and **won't grant anyone else** until the sequence ends.
- Use: atomic read-modify-write (SWP), semaphores, mutexes.
- Recommended: an **IDLE** after the last locked transfer before re-arbitrating.
- Slaves must treat a locked sequence atomically (e.g. a multi-port memory blocks other ports).

SCHEMES COMPARED

Scheme	Pro	Con
Fixed priority	Tiny, predictable for CPU	Starves low priority
Round-robin	Fair, bounded latency	No urgency
Weighted RR	Bandwidth shares	More logic
TDMA	Guaranteed slots	Wastes idle slots

TRAPS

- **Grant ≠ ownership**: HGRANT can change any cycle; it only counts on an edge with HREADY high.
- Using HMASTER (not its registered copy) for the HWDATA mux sends the wrong master's data.
- Master losing grant mid-burst must restart with **NONSEQ**, never resume with SEQ.
- Grant during a wait state: new owner waits for HREADY before driving.
- No handover while HMASTLOCK is high.

ARBITER CHECKS & COVERAGE

```
a_onehot : assert property (@(posedge HCLK) $onehot(HGRANT));

a_lock_hold : assert property (@(posedge HCLK) disable iff (!HRESETn)
  (HMASTLOCK && HREADY) |=> $stable(HMASTER));

// round-robin fairness: a request is granted within N cycles
a_no_starve : assert property (@(posedge HCLK) disable iff (!HRESETn)
  HBUSREQ[i] |-> ##[1:MAX_WAIT] HGRANT[i]);

covergroup cg_handover @(posedge HCLK iff (HREADY && owner_change));
  cp_from : coverpoint prev_master;  cp_to : coverpoint HMASTER;
  x_pair  : cross cp_from, cp_to;           // every master-pair handover
  cp_when : coverpoint ctx;  // enum {MID_INCR, AFTER_FIXED, DURING_WAIT,
AFTER_LOCK}
endgroup
```

INTERVIEW RAPID-FIRE

Why does the data bus switch one cycle after the address bus?
Pipelining: the old master's last data phase overlaps the new master's first address phase.

Default master vs dummy master?
Default: owns the bus when no one asks. Dummy: owns it when all requesters are SPLIT. Both drive IDLE.

How long can a master hold the bus?
Until the arbiter removes grant; a burst or a locked sequence normally completes first.

How do you verify arbiter fairness?
Saturate all requests, track grant counts and max wait per master, assert a bound, cover every handover pair.

Remember: own the bus = HGRANT & HREADY. Address switches first, data one beat later.

SPLIT & RETRY transactions

08

How a slow slave frees the bus in AHB2: the full SPLIT handshake with the arbiter, how RETRY differs, and the deadlock rules behind both.

SPLIT — THE 8-STEP HANDSHAKE

1. Master M2 issues NONSEQ to a slow, split-capable slave S.
2. S sees data won't be ready soon; **records HMASTER = 2** in its split mask.
3. S drives the **two-cycle SPLIT** response (HREADY 0 then 1).
4. Arbiter sees SPLIT and **masks M2's request** — treated as not requesting.
5. Bus is granted to other masters, or to the **dummy master** if everyone is split.
6. S finishes internally and pulses `HSPLITx[2]` for one cycle.
7. Arbiter **unmasks M2**; M2 (still requesting) is re-granted by normal priority.
8. M2 **re-issues the same transfer** (NONSEQ, same addr/ctrl); S completes it with OKAY.

RETRY — SIMPLER, NO ARBITER STATE

- Slave gives two-cycle **RETRY**; nothing is recorded.
- Master **immediately re-issues** the same transfer when it next owns the bus.
- Arbiter keeps normal arbitration: only a **higher-priority** master can slip in.
- Good for short, unpredictable delays (e.g. a FIFO momentarily full).
- Cost: the bus is burned on repeated attempts; low-priority masters can starve.

WHY SPLIT EXISTS

A 50-cycle external memory access with plain wait states blocks every master. SPLIT hands the bus to others while the slave works, raising total bandwidth.

TIMING — SPLIT, RELEASE AND RE-ISSUE (CONDENSED)

Signal	T1	T2	T3	T4 …	Tn	Tn+1	Tn+2
HMASTER	M2	M2	M2	M1 …	M1	M1	M2
HTRANS · HADDR	NONSEQ · A	NONSEQ · B	IDLE	M1 traffic	M1 traffic	M1 last	NONSEQ · A again
HREADY	1	**0**	**1**	…	1	1	1
HRESP	OKAY	**SPLIT**	**SPLIT**	OKAY	OKAY	OKAY	OKAY
M2 request masked	no	no	yes	yes	yes	no	no
HSPLIT[2]	0	0	0	0	1	0	0
HGRANT_M2	1	1	0	0	0	1	1

M2's pending address B is cancelled with IDLE during the second SPLIT cycle. After HSPLIT[2], M2 regains grant and starts again from A — not from B.

DEADLOCK & CORRECTNESS RULES

- A split-capable slave can have at most **one outstanding split per master** — up to 16 entries — and must be able to record all of them.
- Arbiter must have a **dummy master** it can grant when every requester is split, so the bus is always driven.
- `HSPLITx` bits are OR'ed from all split-capable slaves into the arbiter; a bit is only valid for a master that slave actually split.
- A split burst resumes from the split beat with a new **NONSEQ**; earlier beats are already complete.
- HMASTER in the address phase is what the slave records — so HMASTER must be correct even for the dummy master.

TRAPS

- "SPLIT and RETRY both stop the master forever" — **no**: RETRY retries at once; SPLIT waits for HSPLIT.
- HSPLIT is a **pulse**, not a level — an arbiter that samples on the wrong edge loses the release and the master hangs.
- After a SPLIT, the master must **not** deassert HBUSREQ, or it never gets the bus back to re-issue.
- Master must re-issue **the same** address and control — not the next queued transfer.
- Neither exists in AHB-Lite or AHB5.

VERIFICATION PLAN FOR SPLIT/RETRY

- **Checks:** masked master never granted until its HSPLIT bit; re-issued transfer equals the split one; HSPLIT bits only for masters previously split; dummy master drives IDLE.
- **Scoreboard:** a split/retried transfer counts once — on its final OKAY, never on the SPLIT cycle.
- **Liveness:** every split eventually completes (bounded with a timeout assertion).
- **Cover:** SPLIT on single / first beat / mid-burst; RETRY N times in a row; all masters split at once (dummy master path); two HSPLIT bits in the same cycle.
- **Cover:** HSPLIT arriving while the master is still masked vs already re-requesting; SPLIT followed by ERROR on retry.

INTERVIEW RAPID-FIRE

What does the slave store for SPLIT?
The HMASTER number of each split master (a 16-bit mask), plus the transfer it must serve.

What happens if all masters are split?
Arbiter grants the dummy master, which drives IDLE until an HSPLIT arrives.

RETRY or SPLIT for a bridge to a slow bus?
SPLIT — latency is long and predictable-ish; others get the bus meanwhile.

Who decides priority after HSPLIT?
The arbiter, as usual; un-masking does not make the master highest priority.

Remember: RETRY = "try again now"; SPLIT = "go away, I'll call you" (HSPLIT).

Decoder, muxes & multi-layer AHB

09

The interconnect logic that turns masters and slaves into a bus: address decode, the two multiplexers and their select timing, the default slave, and the bus matrix.

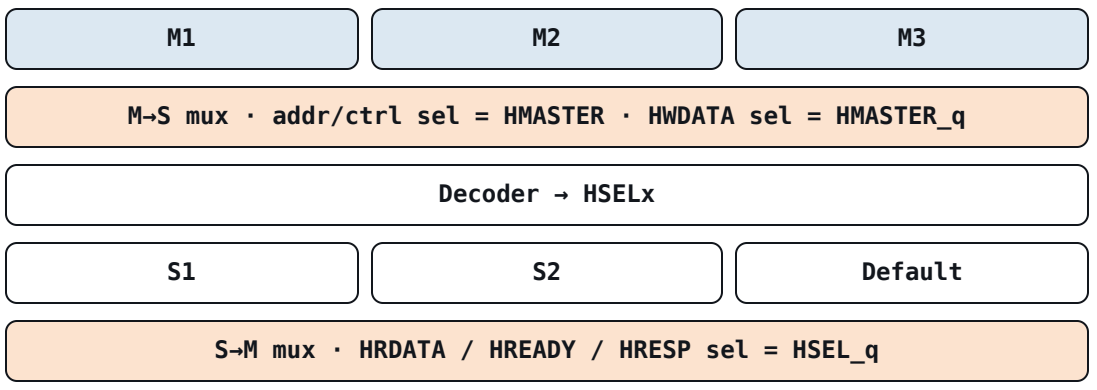
DECODER

- **Combinational** decode of HADDR → one-hot `HSELx` in the address phase.
- Minimum slave region = **1 KB** (that's where the burst rule comes from).
- Unmapped addresses select the **default slave**: zero-wait OKAY for IDLE/BUSY, two-cycle ERROR for NONSEQ/SEQ.
- Decoder also drives the select for the S→M mux (registered — see below).
- Only one HSEL may be high; overlapping regions are a memory-map bug.

EXAMPLE MEMORY MAP

Range	Slave
0x0000_0000 – 0x0003_FFFF	Flash (256 KB)
0x2000_0000 – 0x2000_FFFF	SRAM (64 KB)
0x4000_0000 – 0x4000_FFFF	AHB→APB bridge
0x6000_0000 – 0x6FFF_FFFF	Ext. memory ctrl
everything else	**Default slave → ERROR**

THE TWO MUXES AND THEIR SELECT TIMING




```
// data-phase select: registered only when the bus moves
always_ff @(posedge HCLK or negedge HRESETn)
  if (!HRESETn)    begin sel_q <= DEFAULT; hmaster_q <= '0; end
  else if (HREADY) begin sel_q <= hsel_idx; hmaster_q <= HMASTER; end

// slave-to-master: steered by the slave owning the DATA phase
assign HRDATA = rdata_s[sel_q];
assign HREADY = readyout_s[sel_q];   // also back to every slave
assign HRESP  = resp_s[sel_q];

// master-to-slave: address now, write data one cycle later
assign HADDR  = haddr_m[HMASTER];
assign HWDATA = hwdata_m[hmaster_q];
```

Rule of thumb: **address-phase muxing uses live selects; data-phase muxing uses selects registered on HREADY.** Mixing these up is the #1 interconnect bug.

MULTI-LAYER AHB (BUS MATRIX)

- Each master gets its **own AHB-Lite layer**; a matrix connects layers to slave ports.
- Per slave port: an **arbiter**. Per layer: a **decoder** and an **input stage** that holds a transfer while its slave port is busy (master sees HREADY low).
- Masters accessing **different slaves run in parallel** — e.g. CPU fetching from Flash while DMA writes SRAM.
- Connectivity can be **sparse**: not every master needs every slave.
- Masters stay simple AHB-Lite — no HBUSREQ/HGRANT, no SPLIT.

Matrix	Flash	SRAM0	SRAM1	Bridge
CPU-I	●	●	–	–
CPU-D	●	●	●	●
DMA	–	●	●	●

SHARED BUS VS MATRIX

	Shared AHB2	Multi-layer
Concurrency	1 transfer	1 per slave
Arbiters	1 global	1 per slave port
Masters	Full AHB	AHB-Lite
Area	Small	Grows ~M×S
Latency	Arbitration overhead	Input-stage cycle possible

TRAPS

- Registering HSEL without the **HREADY enable** makes HRDATA come from the wrong slave after a waited transfer.
- Default slave must still give **OKAY to IDLE** — giving ERROR to IDLE is a common bug.
- After IDLE, the S→M mux must point at something with HREADYOUT = 1, or the bus hangs.
- Bus matrix input stage: the master sees a wait state even when the target slave is zero-wait (arbitration lost).

INTERVIEW RAPID-FIRE

Why is the read mux select registered?
HRDATA belongs to the previous address; the decoder already shows the next one.

How to verify a decoder?
Walk every region's first/last byte and one byte past it; cover each HSEL; expect ERROR from holes.

Why multi-layer over AHB2 arbitration?
Parallel paths, simpler masters, no SPLIT complexity.

Remember: live select for address, HREADY-registered select for data.

AHB-Lite & AHB5

10

What AMBA 3 removed to make AHB-Lite, what AMBA 5 added in AHB5 — TrustZone, exclusive access, extended memory types, user signals — and how to answer "which AHB are you verifying?"

SIDE-BY-SIDE

Feature	AHB2 (full)	AHB-Lite (AMBA 3)	AHB5 (AMBA 5)
Masters	Up to 16, global arbiter	One (multi via bus matrix)	One per layer; HMASTER as ID
HBUSREQ / HGRANT	Yes	Removed	Removed
HRESP	2 bits: OKAY/ERROR/RETRY/SPLIT	1 bit: OKAY/ERROR	1 bit + HEXOKAY
SPLIT / RETRY	Yes (HSPLITx)	Removed	Removed
Lock	HLOCKx → arbiter → HMASTLOCK	HMASTLOCK from master	HMASTLOCK + exclusives
HPROT	4 bits	4 bits	7 bits (extended memory types)
Security	–	–	HNONSEC (TrustZone)
Exclusive access	–	–	HEXCL / HEXOKAY
User signals	–	–	HAUSER, HWUSER, HRUSER
Typical CPU	ARM7/ARM9 era	Cortex-M0/M3/M4	Cortex-M23/M33/M55

AHB-LITE IN ONE BREATH

- Single master → **no arbitration**; the master owns the bus all the time and drives IDLE when it has nothing.
- Keeps pipelining, bursts, wait states, ERROR, HPROT, HMASTLOCK.
- Formalises **HREADYOUT** (slave output) vs **HREADY** (muxed input).
- Full AHB slaves that never use SPLIT/RETRY work unchanged on AHB-Lite.
- Multi-master = several AHB-Lite layers into a **multi-layer matrix** (sheet 09).
- Fixed bursts end early only after ERROR.

AHB5 ADDITIONS

- `HNONSEC`: 1 = non-secure transfer; secure slaves reject NS access to secure regions (ERROR).
- `HEXCL` / `HEXOKAY`: exclusive (LDREX/STREX-style) access with a monitor.
- `HPROT[6:0]`: bufferable, modifiable, lookup, allocate, shareable — richer cache attributes.
- `HMASTER`: master ID so the exclusive monitor can tell masters apart.
- Optional properties: endianness (BE8), multi-copy atomicity, signals stable between clocks; later issues add interface parity.
- Terminology moves to **Manager / Subordinate**.

EXCLUSIVE ACCESS — FLOW

1. **Exclusive read** (HEXCL = 1): monitor records address + HMASTER.
2. CPU computes the new value (e.g. semaphore increment).
3. **Exclusive write** (HEXCL = 1) to the same address.
4. No other write hit the address → write performed, `HEXOKAY = 1`.
5. Another master wrote in between → write **not** performed, HEXOKAY = 0 (HRESP still OKAY); software retries.

Exclusive vs locked: locked blocks the bus for everyone; exclusive is optimistic — the bus stays free and only a failed STREX costs a retry.

TRAPS

- A failed exclusive is **not an ERROR** — HRESP is OKAY, HEXOKAY is 0.
- HEXOKAY is meaningful only in the data phase of an exclusive transfer.
- HNONSEC polarity: **1 = non-secure** (easy to invert).
- "AHB-Lite has no locked transfers" — it does: HMASTLOCK.
- HPROT[3] means **modifiable** in AHB5 (was "cacheable").

VERIFICATION DELTAS WHEN THE DUT IS AHB5

- **Security:** cross HNONSEC × region × read/write; expect ERROR for NS→secure, OKAY otherwise.
- **Exclusive:** model the monitor in the scoreboard; cover pass, fail-by-other-master, fail-by-different-address, exclusive to non-exclusive-capable slave.
- **HPROT:** cover each attribute bit; check memory controller policy follows it.
- **User signals:** pass-through checks across bridges and matrices (widths are configurable).

INTERVIEW RAPID-FIRE

Why did ARM create AHB-Lite?

Most MCU systems had one CPU master; dropping arbitration and SPLIT cut area and verification effort.

How does an AHB-Lite system support DMA + CPU?

Multi-layer bus matrix with per-slave arbitration.

What does HEXOKAY = 0 on an exclusive write mean?

The exclusive failed; memory not updated; software retries the sequence.

Can an AHB-Lite master talk to a full AHB system?

Yes, through a wrapper that adds HBUSREQ/HGRANT and handles RETRY/SPLIT.

Remember: Lite subtracts (arbiter, SPLIT, RETRY); AHB5 adds (secure, exclusive, attributes, user).

AHB-to-APB bridge

11

The classic AHB slave project: turning pipelined AHB transfers into two-phase APB accesses, the reference FSM, posted writes, wait-state insertion and error mapping.

ROLE

- An **AHB slave** on one side, the **only APB master** on the other.
- APB is not pipelined: every access = **SETUP** (PSEL=1, PENABLE=0) + **ACCESS** (PENABLE=1) — so the bridge inserts AHB wait states.
- Registers HADDR/HWRITE in the address phase; decodes PSELx for each peripheral.
- AHB bursts become a **sequence of single** APB accesses.
- Writes are **posted**: AHB sees the write finish before the APB access happens.

APB SIGNALS BY VERSION

Signal	Ver	Purpose
PADDR · PWRITE · PWDATA	2	Address / dir / write data
PSELx · PENABLE	2	Select · access phase
PRDATA	2	Read data
PREADY	3	Slave wait states
PSLVERR	3	Slave error
PPROT · PSTRB	4	Protection · byte strobes

ARM REFERENCE BRIDGE FSM (8 STATES)

IDLE
valid read → READ · valid write → WWAIT

READ
APB setup · HREADYOUT = 0 · → RENABLE

RENABLE
APB access · return PRDATA · next valid → READ / WWAIT, else IDLE

WWAIT
HWDATA arrives (AHB write completes) · valid → WRITEP else WRITE

WRITE
APB setup · valid → WENABLEP else WENABLE

WENABLE
APB access · next valid → READ / WWAIT, else IDLE

WRITEP
setup with a transfer pending · HREADYOUT = 0 · → WENABLEP

WENABLEP
access with pending · pending write → WRITEP / WRITE · pending read → READ

"P" = **pending**: another AHB transfer arrived while the posted write was still on APB. Wait states are inserted in READ, WRITEP, and WENABLEP when the next transfer is a read.

TIMING — READ THROUGH BRIDGE

Signal	T1	T2	T3
AHB	NONSEQ rd A	data phase	data phase
State	IDLE	READ	RENABLE
PSEL / PENABLE	0 / 0	1 / 0	1 / 1
HREADYOUT	1	0	1
HRDATA	–	–	PRDATA

Read costs **≥ 1 AHB wait state** (more with PREADY low or slower PCLK).

TIMING — POSTED WRITE

Signal	T1	T2	T3	T4
AHB	NONSEQ wr A	D(A)	free	free
State	IDLE	WWAIT	WRITE	WENABLE
PSEL / PENABLE	0 / 0	0 / 0	1 / 0	1 / 1
HREADYOUT	1	1	1	1

The AHB write is **zero-wait**; APB happens in the background.

DESIGN CONSIDERATIONS

- **Clock ratio:** PCLK = HCLK / n with a `PCLKEN` enable; every APB phase then costs n HCLK cycles.
- **APB3 PREADY low** → bridge holds HREADYOUT low (for reads) or delays the next transfer (writes).
- **PSLVERR** → map to a two-cycle AHB ERROR for reads. For a **posted write** the AHB transfer already completed OKAY — error must go to an interrupt/status register, or posting must be disabled.
- Original ARM bridge: HRESP always OKAY, PRDATA zero outside the access phase (OR-able read mux).
- Ordering: a read after a posted write must wait until the write finishes on APB (read-after-write hazard).

COMMON BRIDGE BUGS

- Capturing PRDATA in SETUP instead of ACCESS.
- Posted write overwritten when a second AHB transfer arrives (missing pending state).
- PADDR/PWRITE changing between SETUP and ACCESS.
- PENABLE asserted without a SETUP cycle, or held high between back-to-back accesses.
- HWDATA sampled in the address phase (one cycle early).

VERIFICATION PLAN & INTERVIEW RAPID-FIRE

Key checks?

APB protocol SVA (setup→access, stable PADDR/PWRITE/PWDATA, PENABLE drop), end-to-end scoreboard AHB→APB per address, ordering of read-after-write.

Key coverage?

Back-to-back RR/RW/WR/WW, bursts into the bridge, PREADY waits 0..N, PSLVERR on read vs write, each PSELx, each clock ratio.

Why is a single write faster than a read?

Writes are posted; reads need the APB data before HREADY can go high.

Bridge vs direct AHB peripheral?

APB cuts power and area for slow register blocks; AHB only where bandwidth matters.

Remember: SETUP then ACCESS; reads wait, writes post; "P" states hold the next transfer.

Verifying AHB in UVM

12

Testbench architecture, the pipelined driver, a monitor that pairs phases correctly, coverage, assertions and the test list — the answer to "walk me through your AHB VIP".

TESTBENCH ARCHITECTURE

```
ahb_test → virtual sequence (+ config_db: active/passive, data width, wait/err knobs)

MASTER AGENT (active)          SLAVE AGENT (reactive)
sequencer                      monitor → req fifo
pipelined driver               responder sequence
monitor                        driver: HREADYOUT/HRESP/HRDATA

SCOREBOARD                     COVERAGE + SVA
memory model by address        covergroups on monitor tx
byte-lane aware                protocol checker bound
drops ERROR'd writes           to the interface

ahb_if (clocking blocks: mst_cb / slv_cb / mon_cb) ⇄ DUT (master, slave, interconnect or bridge)
```

PIPELINED DRIVER — TWO THREADS

```
task run_phase(uvm_phase phase);
  fork addr_phase(); data_phase(); join
endtask

task addr_phase();                      // owns HADDR/HTRANS/ctrl
  forever begin
    seq_item_port.get_next_item(req);
    foreach (req.beat[i]) begin
      drive_ctrl(req, i);               // NONSEQ first, SEQ after
      do @(vif.mst_cb); while (!vif.mst_cb.HREADY);   // accepted
      dq.put(req.beat[i]);              // hand to data phase
    end
    vif.mst_cb.HTRANS <= IDLE;
    seq_item_port.item_done();
  end
endtask

task data_phase();                      // owns HWDATA / samples HRDATA
  forever begin
    dq.get(b);
    if (b.write) vif.mst_cb.HWDATA <= b.data;
    do @(vif.mst_cb); while (!vif.mst_cb.HREADY);   // completed
    b.resp = vif.mst_cb.HRESP;
    if (!b.write) b.rdata = vif.mst_cb.HRDATA;
  end
endtask
```

The mailbox `dq` guarantees data phase N starts on the same edge address phase N+1 starts — exactly AHB's overlap.

MONITOR — PAIR THE PHASES

- On each edge with `HREADY = 1`: first **complete** the pending data-phase item (HRDATA/HWDATA, HRESP), then **capture** a new address phase if `HTRANS` is NONSEQ/SEQ.
- Ignore IDLE/BUSY; track burst beats by NONSEQ→SEQ.
- Non-OKAY: record on the 2nd cycle; mark write data invalid on ERROR.
- Publish one item per **beat** plus one per **burst** via analysis ports.
- Passive mode reuses the same monitor — essential for interconnect/subsystem envs.

REACTIVE SLAVE

Monitor pushes address phases into a FIFO; a forever responder sequence pulls them and returns random waits, OKAY/ERROR and read data from a memory model.

COVERAGE PLAN

Item	Bins / cross
HTRANS transitions	All legal arcs (I→N, N→S, S→B, B→S…)
HBURST × HSIZE × HWRITE	Full cross (illegal size>bus excluded)
Wrap point	Wrap on each beat position
1 KB proximity	Burst ending at 0x3FC / starting 0x400
Wait states	0, 1, 2–4, 5–16 × beat position
HRESP	OKAY/ERROR × first/mid/last beat
Back-to-back	RR, RW, WR, WW, diff. slaves
Byte lanes	HSIZE × HADDR[1:0]
HPROT / HMASTLOCK	Each value; locked sequences

PROTOCOL CHECKER (BIND TO AHB_IF)

- Ctrl and address stable while HREADY low (sheet 03).
- HWDATA stable while write waited.
- No SEQ/BUSY after IDLE; SINGLE never followed by SEQ.
- SEQ address = prev + size (INCR) or wrapped (WRAP).
- HSIZE ≤ bus width; address aligned to HSIZE.
- INCR bursts never cross 1 KB.
- IDLE/BUSY get zero-wait OKAY.
- Non-OKAY is two cycles, same value.
- No X on HTRANS/HREADY/HRESP out of reset.
- Bounded wait (no hang).

TEST LIST

Test	Targets
sanity_rw	Single word write/read-back
all_sizes	Byte/half/word, all lanes
all_bursts	8 HBURST types × sizes
wrap_boundary	Every wrap start offset
kb_boundary	Bursts ending at 1 KB edge
busy_inject	BUSY beats in bursts, INCR ending on BUSY
wait_stress	Random waits, back-to-back
err_mid_burst	ERROR + cancel / continue
default_slave	Holes in memory map
reset_mid_xfer	Reset during burst
random_regress	All knobs, many seeds

BUGS THAT DV INTERVIEWS LOVE

- Monitor samples HRDATA on a waited cycle → false mismatches only under random waits.
- Driver drives HWDATA with HADDR (same cycle) → passes with a lenient slave, fails real RTL.
- Scoreboard compares write data of an ERROR'd write.
- Slave latches address on HSEL without HREADY → wrong transfer after another slave waits.
- Coverage "100%" but no wait on the **last** beat of a burst.
- WRAP address math uses beats only, not beats × size.

INTERVIEW RAPID-FIRE

How do you model pipelining in the driver?

Two forked threads joined by a mailbox: address thread hands accepted beats to the data thread.

Active vs reactive agent?

Master is active (initiates); slave is reactive — it waits for the monitor's request and responds.

Coverage stuck at 92% on WRAP16 × byte?

Check constraints (beats × size, 1 KB), sequence weights, and that the DUT supports byte bursts.

How do you verify early burst termination?

Slave returns ERROR mid-burst; master knob to cancel; scoreboard expects only completed beats.

Remember: two threads, one mailbox; sample only on HREADY; drop ERROR'd writes.